UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RAFAELA

**- Tecnicatura Universitaria en Programación -**

# Gestión de Desarrollo de Software




## Unidad IV: Scrum

**Docente:** Ing. Darío Haspert

**Curso**: 2º año - 2º cuatr.

**< septiembre de 2026 >**

# Índice

UNIDAD 4. Scrum ..... 4
1. Introducción ..... 4
2. Pilares y Valores de Scrum ..... 5
2.1. Pilares de Scrum ..... 5
2.2. Los 5 valores de Scrum ..... 6
3. Roles ..... 7
3.1. Dueño de Producto (*Product Owner*) ..... 7
3.2. Scrum Master ..... 8
3.3. Desarrolladores (*Developers*) ..... 9
4. Artefactos ..... 9
4.1. Backlog de Producto (*Product Backlog*) ..... 10
4.2. Backlog de Sprint (*Sprint Backlog*) ..... 10
4.3. Incremento (*Increment*) ..... 11
5. Eventos ..... 11
5.1. El Sprint ..... 12
5.2. Planificación del Sprint (*Sprint Planning*) ..... 13
5.3. Reunión Diaria (*Daily Scrum*) ..... 14
5.4. Revisión del Sprint (*Sprint Review*) ..... 14
5.5. Retrospectiva del Sprint (*Sprint Retrospective*) ..... 14
5.6. Refinamiento del Backlog (*Backlog Refinement*) ..... 15
5.7. Errores comunes en los eventos ..... 15
6. Cohesión de los eventos ..... 16
7. Scrum Orgánico ..... 16
8. Historias de Usuario e INVEST ..... 17
8.1. Historias de Usuario ..... 17
8.2. El método INVEST ..... 18
9. Estimación ágil: Puntos de Historia y Planning Poker ..... 19
9.1. Puntos de Historia ..... 19
9.2. Planning Poker ..... 19
10. El Tablero de Sprint y el Diagrama de Burndown ..... 20
10.1. Tablero de Sprint ..... 20
10.2. Diagrama de Burndown ..... 20
11. Incremento de Funcionalidad y Definición de Terminado ..... 21

11.1. Incremento de Funcionalidad ..... 21
11.2. Definición de Terminado (*Definition of Done*) ..... 22
12. Técnicas de división de historias ..... 22
12.1. Cortes horizontales vs. cortes verticales ..... 22
12.2. Cuatro patrones principales de división ..... 22
13. Métricas y reportes ágiles ..... 24
13.1. Velocity (Velocidad del equipo) ..... 24
13.2. Burndown Rate ..... 24
13.3. Lead Time y Cycle Time ..... 24
13.4. Sprint Report ..... 25
14. Scrum vs. Kanban: ¿cuándo usar cada uno? ..... 25
15. Resumen ..... 26
Referencias bibliográficas ..... 27
Anexo — Patrones avanzados de división de historias ..... 28

# UNIDAD 4. Scrum

## 1. Introducción

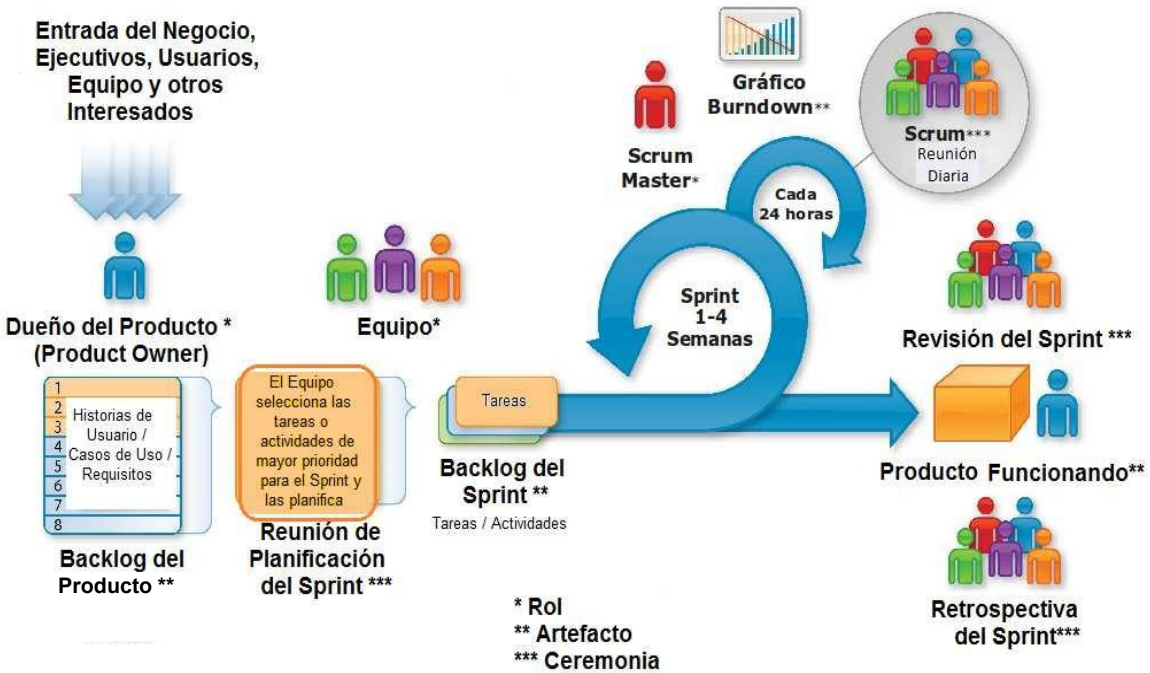
Scrum es el marco de trabajo ágil más utilizado en la industria del software. La mayoría de las personas que dicen trabajar con metodologías ágiles están usando Scrum o alguna variante basada en él.

El nombre proviene de una analogía presentada en un estudio de 1986 por Takeuchi y Nonaka, que comparaba a los equipos de alto rendimiento multidisciplinarios con la formación del *scrum* en el rugby: un grupo compacto que avanza unido hacia un objetivo común. Cuando Jeff Sutherland y Ken Schwaber formalizaron el marco en 1993, adoptaron ese término para capturar la esencia de lo que buscaban: equipos cohesionados, autogestionados y orientados a resultados concretos.

Scrum no es una metodología que prescribe cada paso del proceso de desarrollo. Es un **marco de trabajo** (*framework*) liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos. Se basa en tres roles, cinco eventos y tres artefactos, todos cohesivos entre sí.

La referencia canónica de Scrum es la **Scrum Guide**, publicada y mantenida por Jeff Sutherland y Ken Schwaber a través de Scrum.org. Es un documento público y gratuito, disponible en español, que define con precisión cada elemento del marco. Todo lo desarrollado en esta unidad está alineado con la Scrum Guide en su versión 2020.

Scrum utiliza un proceso **iterativo e incremental**: el trabajo se organiza en ciclos cortos de duración fija llamados *sprints*, en los que el equipo planifica, ejecuta y reflexiona sobre lo realizado antes de comenzar el siguiente ciclo. Esta cadencia genera aprendizaje continuo y permite adaptar el producto a medida que se obtiene información real.



El ciclo de Scrum funciona de la siguiente manera:

- El Dueño de Producto crea y mantiene una lista priorizada de requisitos llamada **Backlog de Producto**.
- Durante la **Planificación** (2 a 4 horas), el Equipo selecciona una porción de esa lista — el **Backlog de Sprint** — y decide cómo implementar esos requisitos.
- El Equipo tiene un tiempo limitado — el **Sprint**, de una a cuatro semanas — para terminar el trabajo comprometido. Cada día se reúne brevemente en la **Reunión Diaria** (15 minutos) para constatar el avance.
- Durante el Sprint, el Scrum Master mantiene al equipo enfocado en el objetivo y elimina los impedimentos que frenan el trabajo.
- Al final del Sprint, el trabajo debe ser **Potencialmente Entregable**: listo para ser enviado al cliente, distribuido o presentado al sponsor.
- El Sprint cierra con una **Revisión** — donde se muestra el incremento — y una **Retrospectiva** — donde el equipo reflexiona sobre su forma de trabajar.
- En el siguiente Sprint, el Equipo selecciona otra porción del Backlog de Producto y el ciclo se repite.

Este ciclo continúa hasta que el Backlog de Producto se haya completado, el presupuesto se haya agotado o se haya alcanzado un hito definitivo del proyecto.

Los Sprints se ejecutan uno tras otro sin tiempo muerto entre ellos: cuando uno termina, el siguiente comienza inmediatamente. Todos tienen la misma duración, acordada al inicio del proyecto y mantenida durante todo el desarrollo.

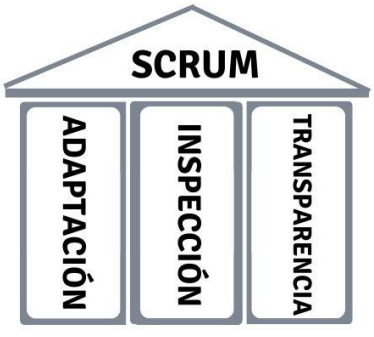
# 2. Pilares y Valores de Scrum

## 2.1. Pilares de Scrum

Existen tres pilares empíricos sobre los que se construye Scrum: **transparencia**, **inspección** y **adaptación**. Un equipo transparente puede inspeccionar su situación real; un equipo que inspecciona con honestidad puede adaptar su plan; un equipo que adapta con agilidad genera valor de manera continua.

**Transparencia**: El proceso y el trabajo emergentes deben ser visibles tanto para quienes realizan el trabajo como para quienes lo reciben. Se observa a través de sus tres artefactos formales. La transparencia permite la inspección. La inspección sin transparencia es engañosa y derrochadora.

**Inspección**: Los artefactos de Scrum y el progreso hacia los objetivos acordados deben inspeccionarse con frecuencia y con diligencia para detectar variaciones o problemas potencialmente indeseables. Scrum proporciona cadencia en forma de sus cinco eventos. La inspección permite la adaptación. La inspección sin adaptación se considera inútil.




**Adaptación**: Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se aplica o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar

una mayor desviación.

Un equipo transparente puede inspeccionar su situación real; un equipo que inspecciona con honestidad puede adaptar su plan; un equipo que adapta con agilidad genera valor de manera continua.

## 2.2. Los 5 valores de Scrum

Scrum no es solo un conjunto de prácticas — tiene una base cultural explícita. La Scrum Guide establece cinco valores que deben guiar el comportamiento del equipo. Sin ellos, los eventos y artefactos de Scrum se convierten en procedimientos vacíos.




**Coraje (*Courage*)**

Los miembros del equipo Scrum tienen el coraje de hacer lo correcto y trabajar en problemas difíciles. Esto implica señalar impedimentos aunque sea incómodo, decir que algo no va a estar listo a tiempo en lugar de prometer lo imposible, y cuestionar decisiones que comprometen la calidad del producto.

**Compromiso (*Commitment*)**

El equipo se compromete con los objetivos del Sprint y con el apoyo mutuo. El compromiso en Scrum no es una promesa de cumplir un número fijo de tareas — es la disposición genuina de hacer todo lo posible para alcanzar el objetivo acordado, adaptándose si las circunstancias cambian.

**Foco (*Focus*)**

Durante el Sprint, el equipo se concentra en el trabajo comprometido y en el objetivo del Sprint. El WIP limitado — que se desarrollará en profundidad en la unidad de Kanban — es una expresión práctica de este valor: hacer menos cosas a la vez para terminarlas mejor y más rápido.

**Respeto (*Respect*)**

Los miembros del equipo Scrum se respetan mutuamente como personas capaces e independientes. El respeto es la base que hace posible la retroalimentación honesta, la colaboración genuina y la resolución constructiva de conflictos.

**Apertura (*Openness*)**

El equipo y sus interesados son transparentes sobre el trabajo y los desafíos que enfrentan. La apertura hace posible la inspección — uno de los tres pilares de Scrum — porque no se puede inspeccionar lo que se oculta.

Estos cinco valores se refuerzan mutuamente y se manifiestan en los tres pilares empíricos sobre los que se construye Scrum: **transparencia**, **inspección** y **adaptación**.

# 3. Roles

Scrum define tres roles que conforman el **Equipo Scrum**: el Dueño de Producto, el Scrum Master y Desarrolladores. Los tres son necesarios y complementarios — ninguno puede reemplazar la función de los otros dos.

Dentro de un Equipo Scrum no hay subequipos ni jerarquías. Es una unidad cohesionada de profesionales enfocados en un objetivo a la vez, el Objetivo del Producto. Los Equipos Scrum son multifuncionales, los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint. También se autogestionan, deciden internamente quién hace qué, cuándo y cómo.

El Equipo Scrum es lo suficientemente pequeño como para seguir siendo ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint, generalmente 10 personas o menos.

## 3.1. Dueño de Producto (*Product Owner*)

El Dueño de Producto es la voz del cliente dentro del equipo. Es responsable de maximizar el valor del producto que el equipo desarrolla, y lo hace gestionando el Backlog de Producto: decidiendo qué se construye, en qué orden y por qué.

Sus responsabilidades principales son:

- Desarrollar y comunicar explícitamente el Objetivo del Producto
- Crear y comunicar claramente los elementos del Product Backlog
- Ordenar los elementos del Product Backlog
- Asegurarse de que el Product Backlog sea transparente, visible y se entienda

El Dueño de Producto es **una sola persona**, no un comité. Puede recibir input de múltiples interesados, pero la decisión final sobre el backlog es suya. Para que el rol funcione, la organización debe respetar sus decisiones — si otros pueden modificar las prioridades del backlog sin pasar por el PO, el rol pierde efectividad.

Habilidades necesarias del PO:

- Identificación de riesgos, priorización, levantamiento de métricas.
- Storytelling (asociado a Historias de usuario o no).
- Habilidades estratégicas como visión de la organización, conocimiento del producto, de las tendencias de la industria/mercado y tecnología.

- Habilidades de negociación y toma de decisiones.
- Habilidades de comunicación para transmitir ideas y alternativas al cliente, al equipo y a todos los implicados.

## 3.2. Scrum Master

El Scrum Master es responsable de establecer Scrum ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo Scrum como de la organización.

Es responsable de lograr la efectividad del Scrum Team, apoyando al equipo en la mejora de sus prácticas, dentro del marco de trabajo de Scrum.

Los Scrum Masters son verdaderos líderes serviciales.

El Scrum Master sirve al **Scrum Team** de varias maneras, que incluyen:

- Guiar a los miembros del equipo en ser autogestionados y multifuncionales
- Ayudar al Scrum Team a enfocarse en crear Increments de alto valor que cumplan con la Definición de Terminado
- Procurar la eliminación de impedimentos para el progreso del Scrum Team
- Asegurarse de que todos los eventos de Scrum se lleven a cabo y sean positivos, productivos y se mantengan dentro de los límites de tiempo recomendados

El Scrum Master sirve al **Product Owner** de varias maneras, que incluyen:

- Ayudar a encontrar técnicas para una definición efectiva de Objetivos del Producto y la gestión del Product Backlog
- Ayudar al Scrum Team a comprender la necesidad de tener elementos del Product Backlog claros y concisos
- Ayudar a establecer una planificación empírica de productos para un entorno complejo
- Facilitar la colaboración de los interesados según se solicite o necesite

El Scrum Master sirve a la **organización** de varias maneras, que incluyen:

- Liderar, capacitar y guiar a la organización en su adopción de Scrum
- Planificar y asesorar implementaciones de Scrum dentro de la organización
- Ayudar a los empleados y los interesados a comprender y aplicar un enfoque empírico para el trabajo complejo
- Eliminar las barreras entre los interesados y los Scrum Teams

El Scrum Master no es un jefe de proyecto tradicional. No asigna tareas, no controla el trabajo individual ni reporta el avance en términos de horas consumidas. Su éxito se mide por la capacidad del equipo para autogestionarse y mejorar continuamente.

Habilidades necesarias del SM:

- Habilidades estratégicas como **visión de la organización**, capacidad de **influencia** y habilidades de **negociación** y **gestión de conflictos**.
- Habilidades de **comunicación** como escucha activa, asertividad, dar y recibir **feedback**.
- Habilidades para la **construcción** y evolución de **equipos de trabajo**.

### 3.3. Desarrolladores (*Developers*)

Los Desarrolladores son el grupo de profesionales que construye el producto. Son **autogestionados** — decide internamente cómo realizar el trabajo — y **multidisciplinario** — tiene todas las habilidades necesarias para entregar un incremento funcional al final de cada Sprint. El trabajo de cada uno de ellos aporta de forma directa a la entrega de valor. Aunque puede haber especialistas, su responsabilidad es compartida como un ente completo. Es deseable que se comparta el conocimiento para evitar centralizarlo en una única persona. Los desarrolladores presentan el resultado de la iteración al Product Owner.

Los desarrolladores siempre son responsables de:

- Crear un plan para el Sprint, el Sprint Backlog;
- Inculcar calidad al adherirse a una Definición de Terminado;
- Adaptar su plan cada día hacia el Objetivo del Sprint; y,
- Responsabilizarse mutuamente como profesionales.

La autogestión del equipo es una consecuencia directa de los valores de Scrum: un equipo con coraje, compromiso y respeto mutuo puede tomar buenas decisiones sin necesitar que alguien les diga cómo hacer su trabajo.

Las habilidades necesarias aunque son variables, dependiendo del entorno de trabajo, destacan:

- Habilidades para **trabajar de forma coordinada, autónoma y responsable**.
- Habilidades **especializadas** según el rol en el equipo.
- Habilidades de **comunicación y feedback**.

## 4. Artefactos

Scrum define tres artefactos que representan el trabajo y el valor del producto en distintos niveles de granularidad. Están diseñados para maximizar la transparencia de la información clave.

Cada artefacto contiene un compromiso para garantizar que proporcione información que mejore la transparencia y el enfoque frente al cual se pueda medir el progreso:

- ✓ Para el **Backlog de Producto**, es el *Objetivo del Producto*.
- ✓ Para el **Backlog de Sprint**, es el *Objetivo del Sprint*.
- ✓ Para el **Incremento** es la *Definición de Terminado*.

## 4.1. Backlog de Producto (*Product Backlog*)

El Backlog de Producto es la **lista única, ordenada y dinámica** de todo el trabajo que se conoce que debe realizarse para mejorar el producto. Es la única fuente de trabajo para el Equipo Scrum — todo lo que el equipo hace surge del Backlog de Producto.

Sus características fundamentales:

- **Único:** no existen listas paralelas de requerimientos. Todo está en el backlog.
- **Ordenado:** los ítems están priorizados de arriba hacia abajo según el valor que aportan al negocio. El ítem con mayor prioridad es el primero de la lista.
- **Dinámico:** evoluciona continuamente. Se agregan ítems cuando surgen nuevas necesidades, se eliminan los que ya no son relevantes y se refinan los que se acercan al tope de la lista.
- **Estimado:** los ítems tienen una estimación de esfuerzo, más precisa cuanto mayor es su prioridad.

El Dueño de Producto es responsable del Backlog de Producto, aunque cualquier miembro del equipo puede proponer ítems. La decisión sobre qué incluir, qué priorizar y qué eliminar es siempre del PO.

Un buen Backlog de Producto es **DEEP**:

- **D**etallado apropiadamente: los ítems de mayor prioridad están más detallados que los de menor prioridad.
- **E**stimado: los ítems que se desarrollarán próximamente tienen estimaciones confiables.
- **E**mergente: el backlog se actualiza continuamente a medida que el equipo aprende.
- **P**riorizado: los ítems están ordenados de manera secuencial, sin empates.

Para lograr un Backlog de Producto *DEEP,* el Equipo y el Dueño de Producto trabajan juntos (con la facilitación del Scrum Master), ejecutando una a varias de las actividades siguientes:

- Depuración de elementos
- Ordenamiento de elementos
- Análisis de requisitos
- División de elementos grandes en varios elementos más pequeños
- Unión de varios elementos
- Estimación o re-estimación de elementos

## 4.2. Backlog de Sprint (*Sprint Backlog*)

El Backlog de Sprint se compone de: el Objetivo del Sprint (*por qué*), el conjunto de ítems del Backlog de Producto que el equipo se comprometió a completar durante el Sprint (*qué*), más el plan de acción para entregar el Incremento (*cómo*).

A diferencia del Backlog de Producto — que pertenece al PO — el Backlog de Sprint es un Plan realizado por y para **los Desarrolladores**. Es una imagen muy visible y en tiempo real del trabajo que los

Desarrolladores planean realizar durante el Sprint para lograr el Objetivo del Sprint. Se actualiza diariamente durante la Reunión Diaria, mostrando en todo momento las tareas pendientes, en curso y terminadas.

Se visualiza típicamente a través de un **Tablero de Sprint**: un tablero físico o digital dividido en columnas que representan los estados del trabajo (Pendiente / En Curso / Terminado), donde cada tarea se representa con una tarjeta o post-it.

### 4.3. Incremento (*Increment*)

Un Incremento es un peldaño concreto hacia el Objetivo del Producto. El Incremento es la suma de todos los ítems del Backlog de Producto completados durante el Sprint más el valor de todos los incrementos anteriores. Al final de cada Sprint, el Incremento debe estar en condiciones de ser utilizado — independientemente de si el Dueño de Producto decide liberarlo o no.

El Incremento debe cumplir con la **Definición de Terminado** acordada por el equipo. Un ítem que no la cumple no forma parte del Incremento.

La Definición de Terminado es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto. Si un elemento del Product Backlog no cumple con la Definición de Terminado, no se puede publicar ni presentar en la Sprint Review. En su lugar, vuelve al Product Backlog para su consideración futura.

## 5. Eventos

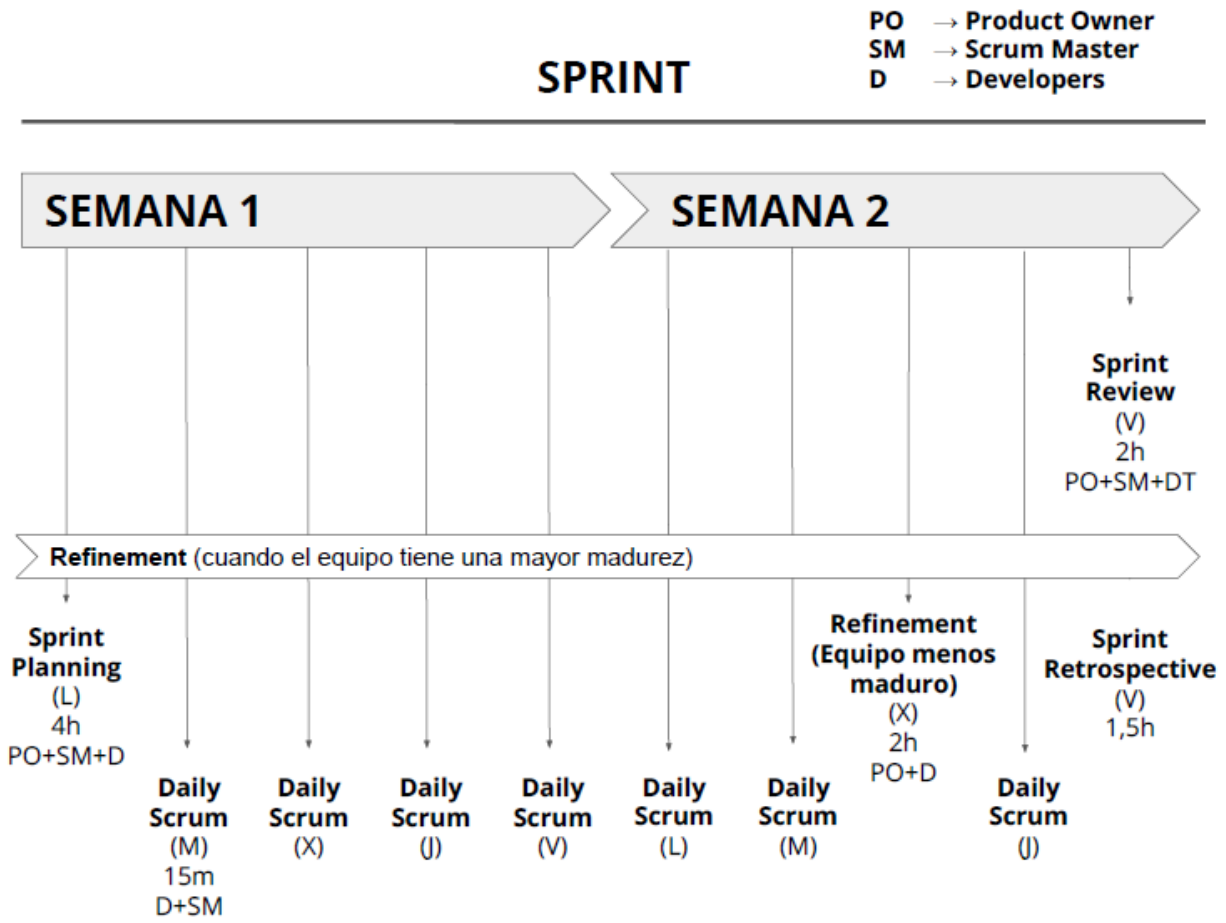
Scrum consta de 5 eventos que ayudan a organizar la estructura de cada iteración. Estos eventos son:

- ✓ Sprint
- ✓ Planificación del Sprint (Sprint Planning)
- ✓ Scrum Diario (Daily Scrum)
- ✓ Revisión del Sprint (Sprint Review)
- ✓ Retrospectiva del Sprint (Sprint Retrospective)

Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos Scrum. Están diseñados específicamente para habilitar la transparencia requerida. Se utilizan en Scrum para crear regularidad y minimizar la necesidad de reuniones no definidas en Scrum.

Lo óptimo es que todos los eventos se celebren al mismo tiempo y en el mismo lugar para reducir la complejidad. Tienen una duración máxima (*timebox*) que se respeta independientemente de si el trabajo está terminado.

**Modelo de Sprint de 2 semanas**




Time-box: hablamos de duraciones máximas de los eventos, todo lo que sea menos, si es suficiente, bienvenido.

## 5.1. El Sprint

El Sprint es el contenedor de todos los demás eventos. Es una iteración de duración fija — entre una y cuatro semanas — durante la cual el equipo transforma un conjunto de ítems del Backlog de Producto en un Incremento potencialmente entregable.

Características del Sprint:

- Todos los Sprints tienen la misma duración. Una vez acordada, no cambia
- No hay tiempo muerto entre Sprints — uno comienza inmediatamente después de que termina el anterior
- No se realizan cambios que pongan en peligro el Objetivo del Sprint
- La calidad no disminuye
- El Product Backlog se refina según sea necesario
- Durante el Sprint, el alcance puede clarificarse y renegociarse entre el PO y el equipo
- Un Sprint puede cancelarse si el objetivo del Sprint se vuelve obsoleto. Solo el Dueño de Producto puede cancelar un Sprint

### *Objetivo del Sprint (Sprint Goal)*

El Objetivo del Sprint es una declaración concisa que resume la finalidad del Sprint — por qué el equipo va a hacer el trabajo que comprometió. Es acordado entre el Dueño de Producto y el Equipo de Desarrollo durante la Planificación.

El Sprint Goal es importante por dos razones. Primero, da coherencia al Sprint: en lugar de ser una lista de tareas sin conexión, el trabajo del Sprint apunta a un resultado concreto. Segundo, da flexibilidad al equipo: si surge un problema que hace inviable completar algún ítem, el equipo puede adaptar el alcance siempre que el objetivo del Sprint siga siendo alcanzable.

Ejemplos de Sprint Goals bien formulados:

- *"Los pacientes pueden registrarse y solicitar un turno médico en línea."*
- *"El administrador puede gestionar el catálogo de productos completo."*
- *"El sistema envía notificaciones automáticas de confirmación de turno."*

Un Sprint Goal mal formulado es simplemente la lista de historias del Sprint: *"Completar las historias HU-01, HU-02 y HU-03"*. Eso no es un objetivo — es un inventario.

## 5.2. Planificación del Sprint (*Sprint Planning*)

La Planificación es el evento que da inicio a cada Sprint. El equipo completo — PO, Scrum Master y Desarrolladores — define qué se va a construir y cómo.

**Duración máxima:** 8 horas para un Sprint de un mes; proporcionalmente menos para Sprints más cortos.

La Planificación aborda 3 temas:

**Tema I - ¿Por qué es valioso este sprint?**

El Product Owner propone cómo el producto podría Incrementar su valor y utilidad en el Sprint actual. Luego, todo el Equipo Scrum colabora para definir un Objetivo del Sprint que comunica por qué el Sprint es valioso para los interesados. El Objetivo del Sprint debe completarse antes de que termine la Sprint Planning.

**Tema II - ¿Qué se puede hacer en este sprint?**

A través de una conversación con el Product Owner, los Desarrolladores seleccionan elementos del Product Backlog para incluirlos en el Sprint actual. El Equipo Scrum puede refinar estos elementos durante este proceso, lo que aumenta la comprensión y la confianza.

Seleccionar cuánto se puede completar dentro de un Sprint puede ser un desafío. Sin embargo, cuanto más sepan los Desarrolladores sobre su desempeño pasado, su capacidad actual y su Definición de Terminado, más confiados estarán en sus pronósticos para el Sprint.

La cantidad de ítems que el equipo puede comprometer se basa en su **Velocidad** (*Velocity*): la cantidad de trabajo que el equipo completó en Sprints anteriores. En el primer Sprint, cuando no hay historial, el equipo hace su mejor estimación y la ajusta en Sprints siguientes.

**Tema III — ¿Cómo se va a hacer el trabajo elegido?**

Para cada elemento del Product Backlog seleccionado, los Desarrolladores planifican el trabajo necesario para crear un Incremento que cumpla con la Definición de Terminado. Esto se hace descomponiendo los elementos del Product Backlog en elementos de trabajo más pequeños de un día o menos. La forma de hacerlo queda a criterio exclusivo de los Desarrolladores. Nadie más les dice cómo convertir los elementos del Product Backlog en Incrementos de valor.

El Objetivo del Sprint, los elementos del Product Backlog seleccionados para el Sprint, más el plan para entregarlos se denominan juntos **Sprint Backlog**.

## 5.3. Reunión Diaria (*Daily Scrum*)

La Reunión Diaria es un evento de 15 minutos, para los Desarrolladores, que ocurre todos los días hábiles del Sprint, siempre en el mismo horario y lugar. Su propósito es inspeccionar el progreso hacia el Objetivo del Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el trabajo planificado entrante. Product Owner o Scrum Master pueden participar si están trabajando activamente en elementos del Sprint Backlog.

Las Daily Scrums mejoran la comunicación, identifican impedimentos, promueven la toma rápida de decisiones y, en consecuencia, eliminan la necesidad de otras reuniones.

La Reunión Diaria no es una reunión de reporte para el Scrum Master o el PO.

## 5.4. Revisión del Sprint (*Sprint Review*)

El propósito de la Sprint Review es inspeccionar el resultado del Sprint y determinar futuras adaptaciones. El Equipo Scrum presenta los resultados de su trabajo a los interesados clave y se discute el progreso hacia el Objetivo del Producto. Es importante obtener feedback sobre el trabajo realizado.

Durante el evento, el Equipo Scrum y los interesados revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. Con base en esta información, los asistentes colaboran sobre qué hacer a continuación. El Product Backlog también se puede ajustar para satisfacer nuevas oportunidades. La Sprint Review es una sesión de trabajo y el Equipo Scrum debe evitar limitarla a una presentación.

**Duración máxima:** 4 horas para un Sprint de un mes.

## 5.5. Retrospectiva del Sprint (*Sprint Retrospective*)

El propósito de la Sprint Retrospective es planificar formas de aumentar la calidad y la efectividad.

El Equipo Scrum inspecciona cómo fue el último Sprint con respecto a las personas, las interacciones, los procesos, las herramientas y su Definición de Terminado. Los elementos inspeccionados suelen variar según el ámbito del trabajo. Se identifican los supuestos que los llevaron por mal camino y se exploran sus orígenes. El Equipo Scrum analiza qué salió bien durante el Sprint, qué problemas encontró y cómo se resolvieron (o no) esos problemas.

El Equipo Scrum identifica los cambios más útiles para mejorar su efectividad. Las mejoras más impactantes se abordan lo antes posible. Incluso se pueden agregar al Sprint Backlog para el próximo Sprint.

La Retrospectiva ocurre después de la Revisión y antes de la próxima Planificación.

**Duración máxima:** 3 horas para un Sprint de un mes.

La Retrospectiva es el mecanismo principal de mejora continua en Scrum. Un equipo que no hace Retrospectivas está condenado a repetir los mismos errores indefinidamente. Una técnica útil y muy conocida para estructurarla es la **Estrella de Mar** (*Starfish*): cinco categorías de observaciones — *Seguir haciendo / Hacer más / Hacer menos / Empezar a hacer / Dejar de hacer* — que organizan la conversación del equipo. Existen muchas más técnicas que ayudan a llevar adelante estos eventos.

## 5.6. Refinamiento del Backlog (*Backlog Refinement*)

El refinamiento del Product Backlog es el acto de dividir y definir aún más los elementos del Product Backlog en elementos más pequeños y precisos. Esta es una actividad continua para agregar detalles, como una descripción, orden y tamaño.

El Refinamiento **no es un evento formal de Scrum** sino una ***práctica recomendada*** que ocurre durante el Sprint, generalmente uno o dos días antes de la próxima Planificación. Su objetivo es preparar el Backlog de Producto para que la Planificación sea eficiente.

Durante el Refinamiento, el equipo y el PO:

- Clarifica y detalla los ítems de mayor prioridad.
- Estima o reestima ítems que van a entrar en los próximos Sprints.
- Divide ítems grandes en partes más pequeñas y manejables.
- Elimina ítems que ya no son relevantes.

Un backlog bien refinado reduce el tiempo de la Planificación y mejora la calidad de los compromisos del equipo.

## 5.7. Errores comunes en los eventos

Un error común en todos los eventos es: NO cumplir los Time-box establecidos para cada uno de ellos.

**Planificación del Sprint**

- Definición de las necesidades no READY.
- Product Owner no disponible.
- Criterio de DONE no completo.

**Reunión Diaria**

- Baja energía / perseguir gente.
- Que se use como una herramienta de supervisión o reporte.
- Hablar de tareas que no estén reflejadas en el panel.

- Entrar demasiado al detalle de los temas.
- Crear conversaciones paralelas.

👁️ **Revisión del Sprint**

- Querer usarlas para justificar un avance parcial.
- Se eternizan.
- No preparadas por el equipo.
- Que cada uno presente su parte.
- No pedir feedback.

🔍 **Retrospectiva**

- Monotonía.
- No guiadas cada vez por una persona.
- No fijar acciones concretas y responsables para mejorar.
- Que solo se traten temas técnicos o que solo se traten las relaciones entre el equipo.
- No revisar los aspectos identificados en la retrospectiva anterior.

## 6. Cohesión de los eventos

Scrum fue diseñado con un enfoque minimalista: cada elemento del marco existe porque cumple una función específica que no puede ser reemplazada por otro elemento. Esto hace que los eventos sean altamente cohesivos entre sí — cuando se elimina uno, se pierde algo que los demás no pueden compensar.

Algunas relaciones de dependencia clave:

**Planificación y Revisión** son una pareja natural: la Planificación define el compromiso del Sprint; la Revisión verifica si ese compromiso se cumplió y qué se aprendió. Sin Revisión, la Planificación pierde su mecanismo de validación.

**Planificación y Reunión Diaria** también se complementan: la Planificación define el plan; la Reunión Diaria monitorea el avance hacia ese plan día a día. Sin Reunión Diaria, los desvíos del plan se detectan tarde.

**Retrospectiva** es el evento más frecuentemente omitido, y su ausencia tiene el costo más alto: sin ella, el equipo no tiene un espacio formal para mejorar su forma de trabajar. Los problemas de proceso se acumulan Sprint tras Sprint sin resolverse.

Eliminar un evento de Scrum no simplifica el marco — lo debilita. Cada evento resuelve un problema específico, y ese problema sigue existiendo aunque el evento no se realice.

## 7. Scrum Orgánico

La adopción completa de Scrum desde el primer día no siempre es posible ni conveniente. Las organizaciones tienen culturas, restricciones y contextos propios que pueden dificultar una implementación inmediata de todos los elementos del marco.

El concepto de **Scrum Orgánico** reconoce que es válido adoptar Scrum de manera gradual e incremental — comenzar con los elementos más accesibles e ir incorporando los demás a medida que el equipo gana madurez y confianza.

Sin embargo, hay principios que no deben perderse de vista en este proceso gradual:

- El objetivo siempre es avanzar hacia la implementación completa del marco, no quedarse cómodo con una versión incompleta.
- La omisión de un evento debe ser una decisión consciente y temporal, no una costumbre permanente.
- El equipo debe entender qué está perdiendo al no implementar cada elemento, para poder evaluar cuándo está listo para incorporarlo.
- La mejora continua aplica también a la adopción del propio marco: cada Sprint es una oportunidad para agregar un elemento que aún no se estaba usando.

# 8. Historias de Usuario e INVEST

## 8.1. Historias de Usuario

Las Historias de Usuario (*User Stories*) son el formato más utilizado para especificar los ítems del Backlog de Producto en Scrum. Describen una funcionalidad desde la perspectiva del usuario que la necesita, en lugar de desde la perspectiva técnica de quien la construye.

El formato estándar es:

*"Como **[rol]**, quiero **[funcionalidad]**, para **[beneficio de negocio]**."*

Ejemplos:

- *"Como paciente, quiero poder solicitar un turno médico en línea, para no tener que llamar por teléfono."*
- *"Como administrador, quiero poder desactivar cuentas de usuario, para gestionar el acceso al sistema."*
- *"Como médico, quiero consultar mi agenda diaria desde cualquier dispositivo, para organizarme mejor entre consultas."*

Este formato obliga al equipo a identificar claramente quién necesita la funcionalidad y qué valor genera — no solo qué hay que construir técnicamente.

Una Historia de Usuario se compone de tres aspectos:

- **Descripción escrita:** el enunciado en formato estándar, que funciona como recordatorio.
- **Conversación:** el diálogo entre el PO y el equipo que clarifica los detalles de la historia.

- **Criterios de aceptación:** las condiciones concretas que deben cumplirse para que la historia se considere terminada.

## 8.2. El método INVEST

INVEST es un acrónimo desarrollado por Bill Wake en 2003 que define seis características que deben tener las Historias de Usuario para ser efectivas:




**I — Independent (Independiente):** Cada historia debe poder planificarse e implementarse sin depender de otra historia. Las dependencias reducen la flexibilidad del equipo para priorizar y complican la planificación. Cuando son inevitables, pueden reducirse combinando o dividiendo historias de manera diferente.

**N — Negotiable (Negociable):** Una historia es una descripción de una necesidad, no un contrato cerrado. Los detalles se acuerdan en la conversación entre el equipo y el PO durante la planificación o el refinamiento. Una historia con demasiados detalles predefinidos limita esa conversación y reduce su utilidad.

**V — Valuable (Valiosa):** La historia debe aportar valor al usuario o al negocio. Una práctica que fomenta este criterio es que las historias las escriban los propios usuarios o el PO, no el equipo técnico.

**E — Estimable (Estimable):** El equipo debe poder estimar el tamaño de la historia con suficiente precisión para priorizarla y planificarla. Si no puede estimarse, suele indicar que la historia está mal definida o que el equipo carece del conocimiento necesario.

**S — Small (Pequeña):** Las historias deben ser lo suficientemente pequeñas para completarse en pocos días dentro del Sprint. Las historias demasiado grandes son Épicas que necesitan dividirse antes de poder comprometerse.

**T — Testable (Comprobable):** La historia debe tener criterios de aceptación claros que determinen cuándo está terminada. Si el PO no sabe cómo probarla, la historia no está suficientemente clara.

**Error frecuente:** usar INVEST como lista de verificación burocrática. INVEST es una guía para la conversación, no un formulario de calidad. Una historia que no supera algún criterio es una señal para conversar más, no para rechazarla automáticamente.

# 9. Estimación ágil: Puntos de Historia y Planning Poker

## 9.1. Puntos de Historia

En Scrum, la estimación del esfuerzo no se mide en horas sino en **Puntos de Historia** (*Story Points*). Un Punto de Historia es una unidad abstracta y relativa que describe cuánto esfuerzo requiere un ítem del Backlog de Producto comparado con otros ítems conocidos.

La estimación relativa tiene ventajas sobre la estimación en horas:

- Las personas son más precisas comparando que midiendo en términos absolutos.
- Evita la presión de comprometerse con horas exactas que luego se convierten en expectativas rígidas.
- Refleja la complejidad, el esfuerzo y la incertidumbre de manera integrada.

La escala de Puntos de Historia es propia de cada equipo. La más utilizada es la **secuencia de Fibonacci modificada**: 1, 2, 3, 5, 8, 13, 21. Los valores no son lineales porque la incertidumbre crece con el tamaño: una historia de 8 puntos no es exactamente el doble de una de 4 — es significativamente más incierta.

## 9.2. Planning Poker

El **Planning Poker** es la técnica de estimación más utilizada en Scrum. Combina la estimación por consenso con un mecanismo que evita el sesgo de anclaje — la tendencia a dejarse influir por la primera estimación que se escucha.

El proceso es el siguiente:

1. El PO presenta un ítem del backlog y responde preguntas del equipo.
2. Cada miembro del equipo elige en privado una carta con su estimación (usando la secuencia de Fibonacci u otra escala acordada).
3. Todos revelan sus cartas simultáneamente.
4. Si hay consenso o las diferencias son menores, se registra la estimación y se avanza.
5. Si hay diferencias significativas, los miembros con las estimaciones más alta y más baja explican su razonamiento. El equipo debate y vuelve a estimar.
6. Se repite hasta alcanzar consenso.

El valor del Planning Poker no está solo en la estimación resultante — está en la conversación que genera. Las diferencias de estimación revelan distintas comprensiones de la historia, supuestos no explicitados o riesgos que algunos miembros identificaron y otros no.

Existen otras técnicas de estimación por consenso — como T-Shirt Sizing, que usa tallas de ropa (XS, S, M, L, XL) en lugar de números, o Wideband Delphi, basada en rondas iterativas de estimación individual seguidas de debate — que responden al mismo principio: obtener una estimación colectiva que incorpore el conocimiento de todos los miembros del equipo.

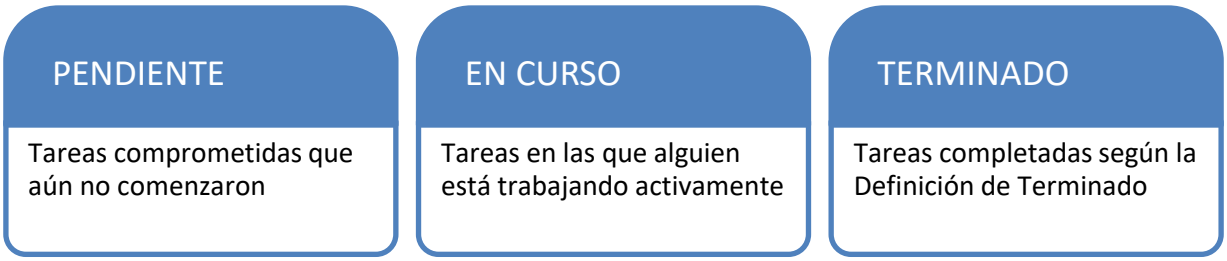
T-Shirt Sizing (XS, S, M, L, XL) es una variante simplificada de estimación relativa que se usa cuando el equipo no tiene experiencia con Puntos de Historia.

# 10. El Tablero de Sprint y el Diagrama de Burndown

## 10.1. Tablero de Sprint

El Tablero de Sprint es la visualización del Backlog de Sprint. Puede ser físico — un panel con post-its — o digital — una herramienta como Trello, Jira o cualquier equivalente. Su función es dar visibilidad inmediata sobre el estado del trabajo a cualquier persona relacionada con el proyecto.

El tablero se divide típicamente en tres columnas:



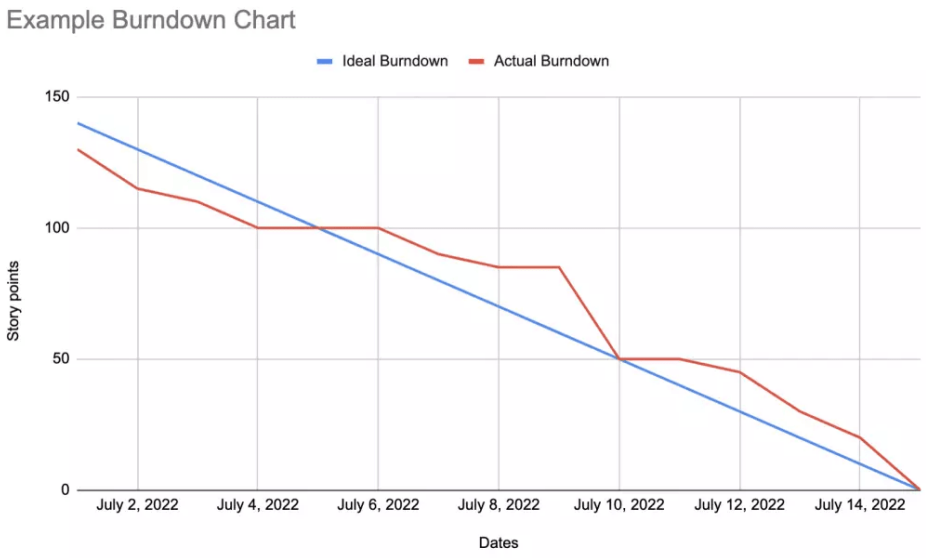

La columna "En Curso" es la más importante de monitorear: un número alto de tareas simultáneamente en curso indica multitasking excesivo y predice problemas de terminación antes del fin del Sprint.

## 10.2. Diagrama de Burndown

El Diagrama de Burndown es el gráfico de seguimiento del Sprint. Muestra cada día cuánto trabajo queda por completar, permitiendo al equipo visualizar si va a cumplir el compromiso del Sprint.

El eje Y representa el esfuerzo restante (en horas o puntos); el eje X representa los días del Sprint. El gráfico incluye dos líneas:

- **Línea ideal:** progresión teórica perfecta, desde el esfuerzo total el día 1 hasta cero el último día.
- **Línea real:** actualización diaria del esfuerzo real restante.




La interpretación del Burndown permite detectar situaciones tempranamente:

| Situación | Interpretación | Acción posible |
|---|---|---|
| **Línea real arriba de la ideal** | El equipo se está atrasando | Revisar impedimentos, reducir alcance si es necesario |
| **Línea real bajo la ideal** | El equipo va adelantado | Considerar agregar ítems del backlog |
| **Línea horizontal** | El esfuerzo no disminuye | Hay tareas bloqueadas o subestimadas |
| **Línea con pendiente ascendente** | El esfuerzo crece | Se están agregando tareas no previstas |

El Burndown debe actualizarse diariamente, idealmente durante la Reunión Diaria, y debe ser visible para todo el equipo.

# 11. Incremento de Funcionalidad y Definición de Terminado

## 11.1. Incremento de Funcionalidad

Al final de cada Sprint, el equipo debe haber producido un **Incremento**: una versión del producto que incorpora el trabajo del Sprint más todo el trabajo de Sprints anteriores, en condiciones de ser utilizada.

El Incremento no es un prototipo ni un trabajo parcial — es software funcionando que cumple con los estándares de calidad acordados. El Dueño de Producto puede decidir no liberarlo al cliente, pero debe estar en condiciones de hacerlo.

### 11.2. Definición de Terminado (*Definition of Done*)

La Definición de Terminado es el acuerdo del equipo sobre qué significa que un ítem está realmente terminado. No es una lista de verificación individual por historia — es un estándar que aplica a todos los ítems del backlog.

Una Definición de Terminado típica en un proyecto de desarrollo web puede incluir:

- El código fue escrito y revisado por otro miembro del equipo.
- Las pruebas unitarias fueron escritas y pasan correctamente.
- La funcionalidad fue probada en el entorno de staging.
- Los criterios de aceptación definidos para la historia fueron verificados.
- La documentación técnica relevante fue actualizada.

La Definición de Terminado crece y se vuelve más exigente a medida que el equipo madura. Un equipo que en su primer Sprint considera terminado algo que simplemente "funciona en la computadora del desarrollador" debería aspirar, en Sprints posteriores, a estándares más rigurosos que incluyan pruebas, revisión de código y documentación.

Un ítem que no cumple la Definición de Terminado **no forma parte del Incremento**. No puede mostrarse en la Revisión como trabajo completado ni contarse en la velocidad del equipo.

## 12. Técnicas de división de historias

### 12.1. Cortes horizontales vs. cortes verticales

Cuando una historia de usuario es demasiado grande para completarse en un Sprint, necesita dividirse. Existen dos enfoques para hacerlo:

**Cortes horizontales:** se divide la historia por capas de arquitectura — base de datos, lógica de negocio, interfaz de usuario. Cada parte se desarrolla independientemente y se integra al final. El problema es que ninguna capa por sí sola genera valor para el usuario: al terminar la capa de base de datos, no hay nada que el PO pueda mostrar ni validar.

**Cortes verticales:** se divide la historia de manera que cada parte atraviese todas las capas de arquitectura y entregue una funcionalidad completa, aunque reducida. Al terminar cada parte, hay algo funcionando que el PO puede mostrar, el usuario puede usar y el equipo puede aprender.

El enfoque ágil privilegia los cortes verticales porque generan feedback temprano y reducen el riesgo de construir algo que nadie necesita.

### 12.2. Cuatro patrones principales de división

**Patrón 1 — Pasos de workflow**

Cuando una historia cubre un proceso de varios pasos, se puede dividir comenzando por el flujo más simple y directo, e incorporando pasos adicionales en historias posteriores.

Ejemplo:

- Historia original: *"Como administrador de contenido, quiero publicar artículos en el sitio corporativo."*
- División:
    - *"...quiero publicar artículos directamente sin validaciones."*
    - *"...quiero publicar artículos con revisión del editor."*
    - *"...quiero publicar artículos con aprobación del área legal."*

**Patrón 2 — Variaciones en reglas de negocio**

Cuando una historia tiene múltiples variantes según reglas complejas, se puede dividir comenzando por la regla más simple y agregando complejidad progresivamente.

Ejemplo:

- Historia original: *"Como usuario, quiero buscar mis operaciones bancarias con fechas flexibles."*
- División:
    - *"...quiero buscar operaciones entre dos fechas específicas."*
    - *"...quiero buscar operaciones de la última semana."*
    - *"...quiero buscar operaciones desde una fecha hasta hoy."*

**Patrón 3 — Operaciones CRUD**

Cuando una historia cubre la administración completa de una entidad, se puede dividir por operación: crear, consultar, modificar, eliminar.

Ejemplo:

- Historia original: *"Como administrador, quiero gestionar los usuarios del sistema."*
- División:
    - *"...quiero crear nuevos usuarios."*
    - *"...quiero consultar la lista de usuarios."*
    - *"...quiero modificar los datos de un usuario."*
    - *"...quiero desactivar un usuario."*

**Patrón 4 — Spike**

Cuando una historia tiene alta incertidumbre técnica y el equipo no puede estimarla con confianza, se crea una **Spike**: una historia de investigación con duración fija cuyo objetivo es obtener información suficiente para estimar y dividir la historia original.

Una Spike no produce funcionalidad — produce conocimiento. Su resultado es una conclusión documentada sobre la viabilidad técnica, el esfuerzo estimado o el enfoque a seguir.

Ejemplo:

- Historia con incertidumbre: *"Como usuario, quiero firmar documentos digitalmente."*
- Spike: *"Investigar las opciones de integración con servicios de firma digital disponibles y estimar el esfuerzo de implementación."* (Duración: 2 días)

Los patrones restantes de Lawrence y la técnica SPIDR completa de Mike Cohn se desarrollan en el **Anexo de esta unidad**.

# 13. Métricas y reportes ágiles

## 13.1. Velocity (Velocidad del equipo)

La Velocity es la cantidad de Puntos de Historia que el equipo completó en un Sprint. Se calcula sumando los puntos de todas las historias aceptadas por el PO al final del Sprint — las historias parcialmente terminadas no cuentan.

La Velocity no es un objetivo a maximizar — es un indicador de capacidad que sirve para planificar. Un equipo con Velocity estable puede comprometerse con más precisión en cada Planificación. Un equipo con Velocity muy variable tiene un problema de proceso que merece ser analizado en la Retrospectiva.

Sus usos principales son:

- **Planificación del Sprint:** el equipo se compromete con una cantidad de trabajo consistente con su Velocity histórica.
- **Predicción de entregas:** dividiendo el trabajo restante del backlog por la Velocity promedio, se puede estimar cuántos Sprints faltan para completar el producto.
- **Detección de problemas:** una caída sostenida en la Velocity puede indicar deuda técnica acumulada, cambios en el equipo o impedimentos no resueltos.

## 13.2. Burndown Rate

El Burndown Rate es la velocidad con que el equipo reduce el esfuerzo restante durante el Sprint. Se visualiza en el Diagrama de Burndown desarrollado en la sección 10.

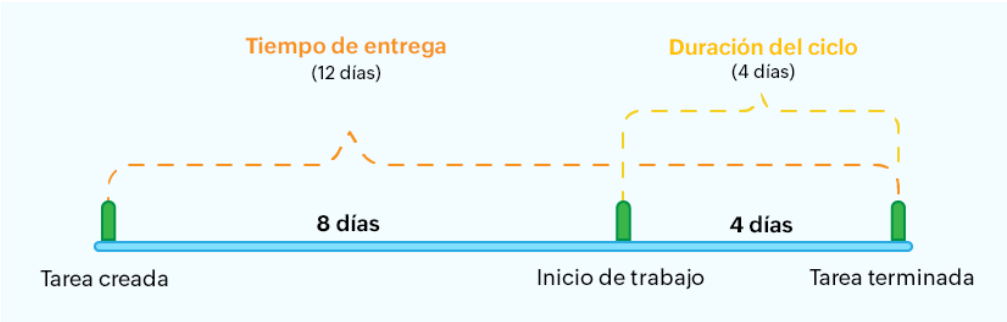
Un Burndown Rate consistente con la línea ideal indica que el equipo trabaja a un ritmo sostenible. Las desviaciones — hacia arriba o hacia abajo — son señales que el equipo analiza en la Reunión Diaria y en la Retrospectiva.

## 13.3. Lead Time y Cycle Time

El **Cycle Time** (tiempo de ciclo) mide el tiempo que el equipo tarda en completar un ítem desde que comienza a trabajar en él hasta que lo entrega. Es una medición de la eficiencia del proceso de desarrollo.

El **Lead Time** (tiempo de entrega) mide el tiempo total desde que el ítem es solicitado hasta que es entregado, incluyendo el tiempo de espera en el backlog. Es lo que percibe el cliente.

La diferencia entre Lead Time y Cycle Time revela cuánto tiempo los ítems esperan en el backlog antes de que el equipo comience a trabajar en ellos. Un Lead Time mucho mayor que el Cycle Time indica un backlog sobredimensionado o una capacidad del equipo insuficiente para la demanda.




### 13.4. Sprint Report

El Sprint Report es el informe que se genera al cierre de cada Sprint, típicamente durante la Revisión. Proporciona una visión completa del Sprint para los interesados.

Contenido típico de un Sprint Report:

| Sección | Contenido |
|---|---|
| **Objetivo del Sprint** | La meta acordada en la Planificación |
| **Trabajo completado** | Historias terminadas con sus puntos de historia |
| **Trabajo no completado** | Historias que no se terminaron y razones |
| **Velocity del Sprint** | Puntos completados vs. Velocity histórica |
| **Impedimentos encontrados** | Obstáculos que afectaron el Sprint |
| **Acciones de mejora** | Compromisos de la Retrospectiva para el próximo Sprint |

El Sprint Report debe ser claro, conciso y comprensible para interesados no técnicos. Los problemas se describen objetivamente sin señalar individuos. Las acciones de mejora deben ser específicas y medibles.

## 14. Scrum vs. Kanban: ¿cuándo usar cada uno?

Scrum y Kanban son los dos marcos ágiles más utilizados, pero responden a necesidades distintas. Elegir entre ellos — o combinarlos — es una decisión que el equipo debe tomar con criterio.

| Dimensión | Scrum | Kanban |
|---|---|---|

| | | |
|---|---|---|
| **Estructura temporal** | Iteraciones fijas (Sprints) | Flujo continuo sin iteraciones fijas |
| **Roles definidos** | Sí: PO, Scrum Master, Equipo | No define roles específicos |
| **Compromisos** | El equipo se compromete con un Sprint Goal | No hay compromisos formales por período |
| **Cambios durante el ciclo** | El alcance del Sprint no cambia | Los ítems pueden cambiar en cualquier momento |
| **Métricas principales** | Velocity, Burndown | Cycle Time, Throughput, WIP |
| **Mejor para** | Proyectos con entregas definidas por Sprint y equipo estable | Flujos continuos de trabajo, mantenimiento, soporte |
| **Curva de aprendizaje** | Mayor — requiere adoptar roles y eventos | Menor — se puede implementar sobre el proceso existente |

**Cuándo elegir Scrum:**

- El proyecto tiene un alcance definible en incrementos.
- El equipo trabaja junto en el mismo producto.
- Se puede comprometer trabajo por períodos fijos.
- Se busca estructurar un equipo que está aprendiendo a trabajar de manera ágil.

**Cuándo elegir Kanban:**

- El trabajo es continuo y variable en tipo y tamaño.
- Los ítems llegan de manera impredecible y deben atenderse con rapidez.
- El equipo gestiona mantenimiento o soporte junto con desarrollo.
- Se busca una adopción gradual sin restructurar los roles del equipo.

**Scrumban:** en la práctica, muchos equipos combinan elementos de ambos marcos. Una combinación frecuente es usar la estructura de Sprints de Scrum con el WIP limitado y el flujo visual de Kanban. A esta combinación se la conoce informalmente como **Scrumban**.

## 15. Resumen

Scrum es un marco de trabajo iterativo e incremental que organiza el trabajo en Sprints de duración fija. Sus tres roles — Dueño de Producto, Scrum Master y Equipo de Desarrollo — tienen responsabilidades complementarias y no intercambiables. Sus tres artefactos — Backlog de Producto, Backlog de Sprint e Incremento — representan el trabajo en distintos niveles de granularidad. Sus cinco eventos —

Planificación, Reunión Diaria, Revisión, Retrospectiva y Refinamiento — crean la cadencia de inspección y adaptación que hace posible la mejora continua.

Los cinco valores de Scrum — Coraje, Compromiso, Foco, Respeto y Apertura — son la base cultural del marco. Sin ellos, los eventos y artefactos se convierten en procedimientos sin propósito.

Las métricas de Scrum — Velocity, Burndown, Cycle Time y Lead Time — no son objetivos sino herramientas de aprendizaje. Su valor está en revelar tendencias y motivar conversaciones sobre cómo mejorar.

En la unidad siguiente se desarrolla Kanban, el segundo gran marco ágil, con sus propias prácticas, métricas y criterios de aplicación.

## Referencias bibliográficas

Schwaber, K. y Sutherland, J. (2020). *The Scrum Guide*. Scrum.org. Recuperado de https://scrumguides.org

Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.

Wake, B. (2003). *INVEST in Good Stories, and SMART Tasks*. XP123. Recuperado de https://xp123.com

Rubin, K. S. (2012). *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley.

# Anexo — Patrones avanzados de división de historias

*(Lectura complementaria)*




**Patrones adicionales de Richard Lawrence**

**Patrón 5 — Mayor esfuerzo concentrado**

Cuando una historia tiene una parte de mucho mayor esfuerzo que el resto, se puede comenzar por esa parte y agregar las demás en historias posteriores.

Ejemplo:

- Historia original: *"Como usuario, quiero pagar con VISA, MasterCard y American Express."*
- División: comenzar con la infraestructura de pago electrónico (que aplica a las tres), luego agregar cada tarjeta como historia separada.

**Patrón 6 — Variaciones en datos**

Cuando una historia debe manejar múltiples formatos o tipos de datos, se puede comenzar con el más común o más simple e incorporar los demás progresivamente.

**Patrón 7 — Interfaz de usuario**

Cuando la complejidad está en la interfaz más que en la funcionalidad, se puede implementar primero la funcionalidad con una interfaz mínima y mejorar la experiencia de usuario en historias posteriores.

**Patrón 8 — Diferir performance**

Cuando optimizar el rendimiento requiere esfuerzo importante, se puede implementar primero la funcionalidad con performance aceptable e invertir en optimización en una historia posterior.

**Patrón 9 — Complejo/Simple**

Cuando una historia tiene múltiples escenarios de complejidad creciente, se comienza por el escenario más simple y se agregan los demás progresivamente.

**S.P.I.D.R. de Mike Cohn**

Por su parte, Mike Cohn propone cinco patrones de división bajo el acrónimo SPIDR:

- **Spike:** investigación con duración fija para reducir incertidumbre (desarrollado como Patrón 4 en la sección principal).
- **Paths (Caminos):** dividir por flujos o secuencias de pasos dentro de la historia.
- **Interfaces:** dividir por tipo de interfaz o dispositivo, comenzando por la versión más simple.
- **Data (Datos):** dividir por tipos o formatos de datos, comenzando por el más frecuente o crítico.
- **Rules (Reglas):** dividir por validaciones o reglas de negocio, incorporándolas progresivamente.

Adicionalmente, Cohn sugiere prácticas complementarias como dividir por tipo de usuario, diferir la arquitectura en etapas tempranas del producto, y "mockear" integraciones con sistemas externos para desbloquear el avance mientras se desarrolla la interfaz formal.